Innovation Center for Education

NCS
NihonCommunicationSolutions

THE YENEPOYA INSTITUTE OF ARTS SCIENCE COMMERCE AND MANAGEMENT

PROJECT-LOW LEVEL DESIGN

On

Identity Security Framework For Edtech

NAME : Khadeejath Mufeeda Kunhadukkam

GUIDED BY:

REG NO : 23BSCED002

TABLE OF CONTENT :

1. Introduction ---------- **1**

1.1 Purpose and Scope ---------- **2**
1.2 Project Executive Summary ---------- 2
1.3 Constraints ---------- 2
1.4 Document Organization ---------- 3

2. System Overview ---------- **3**

3. Module Design ---------- **4**

3.1 User Management Module
3.2 Verification Module
3.3 Access Control Module
3.4 Session Management Module
3.5 Logging and Monitoring Module

4. Data Flow Design ---------- **5**

5. Database Design ---------- **5**

5.1 User Table ---------- **5**
5.2 Verification Table ---------- **5**
5.3 Log Table ---------- **6**

6. User Interface Design ---------- **6**

7. Security Implementation ---------- **6**

8. Testing and Validation ---------- **6**

9. Project References ---------- **7**

10. Glossary ---------- 7

11. Front-end ---------- **8**

12.Back-end ---------- **9**

1. INTRODUCTION

1.1 Purpose and Scope

Purpose:

This document provides a detailed low-level design of a secure identity and access management system tailored for an EdTech platform. The goal is to design a system that ensures secure authentication, controlled access, and proper monitoring of user activities.

The system focuses on maintaining the integrity, confidentiality, and availability of user identity data within an online learning environment.

Scope:

The system covers:

- Multi-user authentication system
- Access control based on user roles
- Session tracking and user monitoring
- Logging and auditing of system activities
- OTP verification through email

In Scope:

- Login and registration system
- Email-based verification
- Role-based dashboards (Admin, Faculty, Student)
- Session handling and timeout
- Logging user interactions

Out of Scope

- Course content delivery system
- Online exams or grading modules
- Payment processing
- Integration with external education APIs

1.2 Project Executive Summary

Modern EdTech platforms handle sensitive user data including personal details and learning records. Ensuring secure identity management is critical to prevent unauthorized access and misuse.

This project introduces a structured identity management system that:

- Authenticates users using secure credential validation
- Adds an additional verification layer through email-based confirmation
- Implements role-based permissions to restrict access
- Tracks and records user activities for audit purposes
- Maintains secure sessions for active users

The system is implemented using:

- **Frontend:** HTML, CSS (Web Interface)
- **Backend:** Flask (Python Framework)
- **Database:** SQLite

This solution enhances system reliability and ensures controlled access within the EdTech platform.

1.3 Constraints

Technical Constraints

- Limited scalability due to SQLite database
- Flask-based backend suitable for small to medium applications
- Email verification depends on external SMTP configuration
- Handling multiple simultaneous sessions may affect performance

Security Constraints

- Protection of user credentials is mandatory
- Sessions must be protected from hijacking
- Input validation must prevent malicious inputs

Operational Constraints

- Designed for academic/demo purposes
- Limited real-time deployment testing
- Requires stable internet for email verification

1.4 Document Organization

The document is structured into:

1. Introduction
2. System Overview
3. Module Design
4. Data Flow
5. Database Structure
6. Interface Design
7. Security Implementation
8. Testing Strategy

2. System Overview

The system is designed using a **web-based architecture**:

- Users interact through browser interface
- Backend server processes logic and authentication
- Database stores user and system data

The system ensures that each request is verified before granting access.

3. Module Design

3.1. User Management Module

- Handles user registration
- Stores user credentials securely
- Assigns roles to users

3.2. Verification Module

- Sends verification code to email
- Confirms identity before granting access
- Prevents unauthorized login attempts

3.3. Access Control Module

- Implements role-based permissions
- Restricts access to specific pages
- Ensures users access only allowed resources

3.4. Session Management Module

- Maintains active user sessions
- Handles login and logout
- Implements session timeout

3.5. Logging and Monitoring Module

- Records user actions
- Tracks login attempts
- Provides logs for administrative review

4. Data Flow Design

i. User submits registration/login form
ii. Backend validates input
iii. Verification code is generated and sent
iv. User enters verification code
v. System validates code
vi. Session is created
vii. User accesses system features
viii. Actions are logged

5. Database Design

Tables Used

User Table

- user_id
- username
- email
- password (encrypted)
- role

Verification Table

- id
- user_id
- verification_code
- expiry

Log Table

- log_id
- user_id
- activity
- timestamp

6. User Interface Design

The system includes:

- Signup Page
- Login Page
- Verification Page
- Role-based Dashboard

The interface is designed to be simple and easy to use.

7. Security Implementation

- Password encryption before storage
- Email-based verification system
- Role-based access restrictions
- Session protection mechanisms
- Input sanitization to prevent attacks

8. Testing and Validation

- Functional testing of login and verification
- Role-based access testing
- Session handling validation
- Basic security testing (input validation, authentication checks)

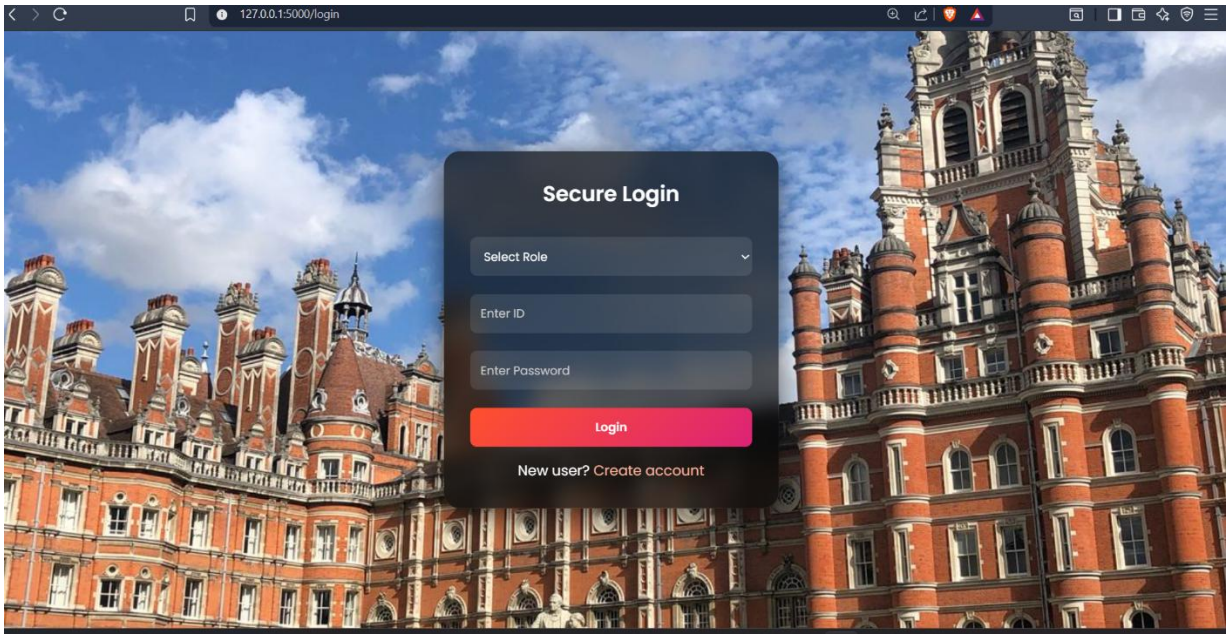
1.6 Project References

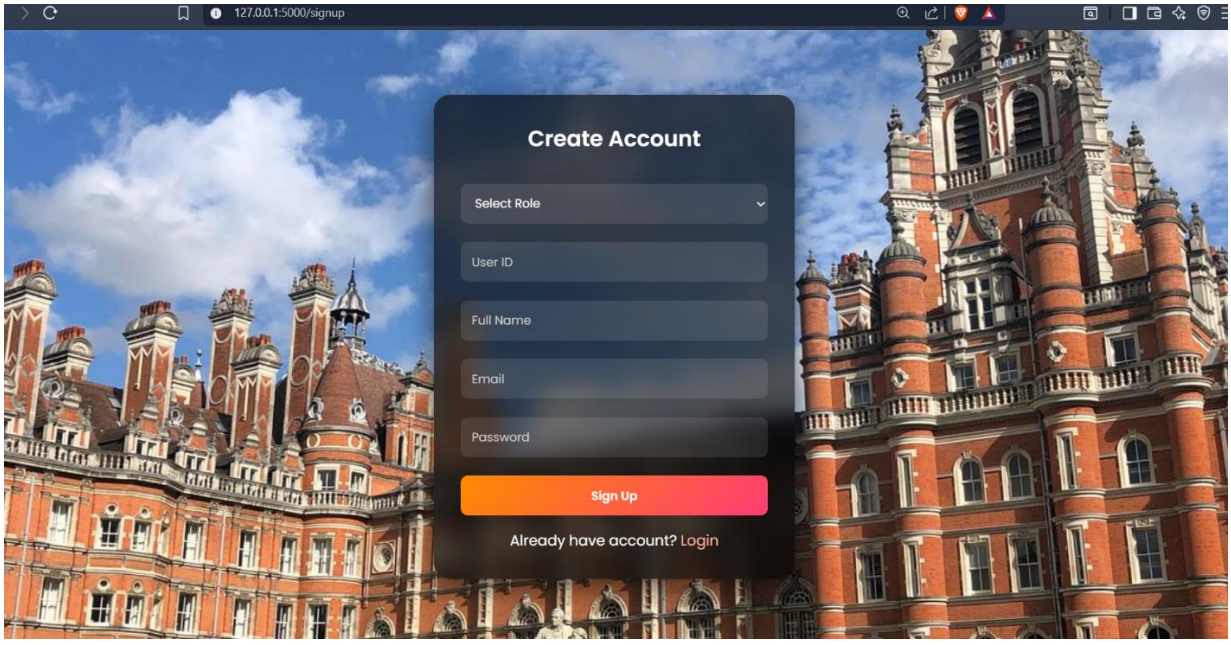
- Flask Official Documentation
- SQLite Database Documentation
- OWASP Top 10 Security Guidelines
- figma

1.7 Glossary

- RBAC**:** Role-Based Access Control
- Authentication**:** Verifying user identity
- Authorization**:** Granting access permissions
- Session**:** Active user login state
- Encryption**:** Securing data
- Verification **Code:** Temporary code for user validation

FRONT-END


127.0.0.1:5000/login
Secure Login
Select Role
Enter ID
Enter Password
Login
New user? Create account


127.0.0.1:5000/signup
Create Account
Select Role
User ID
Full Name
Email
Password
Sign Up
Already have account? Login

BACK-END – FLASK

app.py > verify_otp

```python
from flask import Flask, render_template, request, redirect, session, send_from_directory
import sqlite3
import random
import smtplib
import os
from datetime import datetime, timedelta
from email.mime.text import MIMEText
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename

app = Flask(__name__)
app.secret_key = "secretkey"

# ---------------- SESSION TIMEOUT ----------------
app.config["SESSION_PERMANENT"] = True
app.config["PERMANENT_SESSION_LIFETIME"] = timedelta(minutes=5)

# ---------------- FILE UPLOAD CONFIG ----------------
UPLOAD_FOLDER = "uploads"
ALLOWED_EXTENSIONS = {"pdf","docx","pptx","txt","png","jpg","jpeg"}
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

def allowed_file(filename):
    return "." in filename and filename.rsplit(".",1)[1].lower() in ALLOWED_EXTENSIONS

# ---------------- EMAIL CONFIG ----------------
EMAIL_ADDRESS = "identityedtechproject@gmail.com"
EMAIL_PASSWORD = "uvntlsoezanuwbtc"

# ---------------- ADMIN INFO ----------------
ADMIN_ID = "admin001"
ADMIN_PASSWORD = "admin123"
ADMIN_NAME = "System Admin"
ADMIN_EMAIL = "mufei.dha@gmail.com"

# ---------------- DATABASE ----------------
def get_db():
    conn = sqlite3.connect("edtech.db")
    conn.row_factory = sqlite3.Row
    return conn

# ---------------- EMAIL FUNCTION
```